Cycle-Block Kernels for Positive Handle Cancellation

Six-state path compression, binary repetition,
weighted deferred forests, and fill-free transport

Research continuation for Vladimir Reshetnikov's ProveIt project
Developed with ChatGPT

12 October 2026

Abstract

The current unknot-recognition programme has source-authenticated handle data and several exact local cancellation engines, but its general quasi-polynomial complexity target still requires geometric and global bounds. We develop a complementary local optimizer whose parameter is the *largest cycle rank of an individual biconnected block*, rather than the total number of cycles or a supplied tree decomposition.

For a nonnegative occurrence matrix with nonnegative row weights, let G be its pivot-capable bipartite support graph and let $\kappa = \max_B(|E(B)| - |V(B)| + 1)$. We give a self-contained exact algorithm with bit complexity $S^{O(1)}2^{O(\kappa)}$. Its central reduction replaces every degree-two path by at most six weighted boundary states. A cyclic block of rank $b \geq 2$ has at most $3b - 2$ such paths after retaining its parent articulation. Block interfaces need only one matching-capacity bit. Supplied repeat-safe alternating paths admit six-by-six max-plus powers, independent complete-table certificates, and computation polynomial in their binary lengths.

We also extend the project's deferred-forest argument to weighted partial cancellation, give a linear-size positive transport circuit that avoids quadratic passive fill, and exhibit connected rank-one-block families with arbitrarily large optimal residuals. The latter separate fast local optimization from geometric completion. The package includes a standard-library implementation, 50 passing test methods, 31,581 exhaustive matrix-case roles, 4,800 forest comparisons, reproducible component measurements, and a checked symbolic cycle with 2^{1025} edges. These results do not constitute a general quasi-polynomial unknot recognizer. Classical matching results and inherited ProveIt arguments are explicitly separated from this continuation's block/path refinement.

Source pin. `d9456c5bdb94820bfa9e480e9ed73c10a769e59a`.
Deliverable status. Mathematical arguments and executable research kernels; no live repository change, native geometric promotion, or proof-assistant verification.

Contents

1 Research target, current baseline, and contribution boundaries **4**
1.1 The actual complexity target . . . 4
1.2 What the inspected predecessor work already supplies . . . 4
1.3 The new refinement . . . 5

2 Unsigned cancellation, private orders, and matching **5**
2.1 Input and objective . . . 5
2.2 Why positive fill cannot eliminate more rows . . . 6
2.3 The pivot-capable support graph . . . 7

3 Exact composition over biconnected blocks **8**

4 Six-state compression of degree-two paths **9**
4.1 Boundary information that a path actually exposes . . . 9
4.2 Special cases that must not be dropped . . . 10

5 The cycle-block kernel theorem **10**
5.1 A small branching skeleton . . . 10
5.2 Preprocessing and witness accounting . . . 12

6 Binary repetition and complete six-state certificates **12**
6.1 A typed transition algebra . . . 12
6.2 Certifying the maximum, not only a realizing path . . . 13
6.3 What a binary summary does not output . . . 14

7 Weighted partial cancellation without primal-forest search **15**
7.1 Why saturation is insufficient for this contract . . . 16
7.2 A concrete forest-choice witness . . . 16

8 Linear-size positive transport circuits **16**
8.1 An immutable triangular dependence structure . . . 16
8.2 A genuine output-size separation . . . 17
8.3 Numerical transport is not attaching-curve transport . . . 18

9 Residual barriers and comparison with other parameters **18**
9.1 Easy optimization can leave an arbitrarily large core . . . 18
9.2 The parameter is local but not universally superior . . . 19
9.3 A safe local portfolio . . . 19

10 Integration contract and a conditional route to quasi-polynomial cost **20**

10.1 The maintained source boundary 20
10.2 Executed implementation and configured resource limits 20
10.3 The actual global proof obligation 21

11 Executed verification and component measurements 22
11.1 Environment and independence of the checks 22
11.2 Exhaustive matrix-case roles 22
11.3 Weighted, typed, and forest audits 22
11.4 Paired matrix-optimization measurements 23
11.5 Equal-query passive transport measurements 24
11.6 Complete binary-power queries 24

12 Further research questions and concrete next experiments 25

13 Conclusion 27

A Reproduction and API contracts 27

B Claim ledger and source audit 28

1 Research target, current baseline, and contribution boundaries

1.1 The actual complexity target

Lackenby's March 2021 slides state an announced unknot-recognition bound $2^{O((\log n)^3)}$, a quasi-polynomial bound [1]. This should not be silently strengthened to $n^{O(\log n)}$ when describing that particular announcement. His July 2026 preprint, *Incompressible surfaces, hierarchies and unknot recognition*, presents a hierarchy-based algorithm and distinguishes iteration counting from the running times of its individual operations [2]. In Section 9 the relevant iteration estimate is $L(g+1)^L$, with L a hierarchy-length bound and g a pattern-complexity bound. This makes compressed local operations relevant, but does not by itself give their costs or a general quasi-polynomial bound for an implementation. We use this distinction throughout.

The inspected ProveIt continuation already contains a faithful binary-affine description of raw normal cuts in every cell dimension, including source charts. Its description is not merely a list of cellular boundary matrices. The maintained report explicitly separates this construction from parallelity removal, bounded admissible cores, marked successor transport, and global hierarchy analysis [6]. Accordingly, this article does not attempt to replace embedded three-dimensional attaching data by an abstract matrix.

Our chosen subproblem begins *after* an authenticated handle core has supplied its nonnegative occurrence counts. It asks which two-handles can be cancelled by the specified positive-singleton calculus while minimizing the weight of the genuinely unresolved two-handles. A fast and optimal answer improves one local stage; it does not determine whether the unresolved part is a ball, handlebody, or knot exterior of a particular type.

1.2 What the inspected predecessor work already supplies

The maintained `core_handlebody.py` uses raw one/two-handle intersections, not signed exponent sums. Its private-column queue is complete for full elimination in the unsigned positive calculus, and counts are saturated at two for that feasibility test [5]. Report 105 proves deferred primal-forest selection for full raw cancellation using uncapped source-cycle parity [7]. Reports 109 and 110 develop weighted deferral, private/positive equivalence, ordered incidence interfaces, and repeat-safe transfer ideas [9, 10].

We therefore do *not* claim to originate any of the following: the positive cancellation model; its stopping-set obstruction; the relation to uniquely restricted matchings; the general deferred-forest principle; or the use of max-plus tables for repeated bounded interfaces. These are prerequisites, restated and proved where necessary to make the new algorithm independently readable.

Uniquely restricted matchings are classical. Golumbic, Hirst, and Lewenstein introduced the concept and already gave a linear-time maximum-cardinality algorithm on cacti [3]. In particular, the cactus specialization below is *not* a first tractability result for cacti. Our formulation handles unit eligibility, binary row weights, attached-block response values, certified binary path powers, and the native positive-cancellation interpretation. Recent general-graph parameterized work gives a treewidth algorithm with a quadratic-width exponent [4]; the inspected project reports use more specialized ordered bipartite interfaces. We do not claim to improve all of these algorithms on all inputs.

1.3 The new refinement

The principal contribution is a direct route from a large cycle-bearing block to a small *branching skeleton*. Degree-two paths carry exactly the information relevant to both matching capacities and directed alternating cycles. At most six states suffice per path; an individual block of cycle rank $b \geq 2$ needs at most $3b - 2$ paths. Different blocks communicate through one capacity bit at a cut vertex. Thus the exponential factor depends on the *largest individual block rank*, not the sum of all ranks.

A second contribution is a precise positive transport circuit. Once a source-private order is known, all passive numerical transport can be represented without performing its potentially quadratic fill. This is useful independently of whether the block optimizer improves the selected set. The partial weighted forest theorem then shows that, under the genuine source parity contract, optimizing this matrix does not require enumeration of all primal spanning forests.

All proofs are mathematical proofs under explicit hypotheses, not formally checked proofs. The finite audits test implementations and boundary cases; they do not substitute for proofs. Novelty is claimed only as a refinement of the inspected continuation, not as an exhaustive priority determination over the entire matching literature.

2 Unsigned cancellation, private orders, and matching

2.1 Input and objective

Let R be a finite set of row labels and C a finite set of column labels. The input is a sparse matrix

$$A = (A_{rc}) \in \mathbb{N}^{R \times C}, \qquad w : R \longrightarrow \mathbb{N}.$$

A row represents a genuine source two-handle; a column represents a source one-handle. Zero rows remain rows. The weights express a chosen nonnegative optimization objective, not a topological invariant. A zero-weight row need not be cancelled in an optimum.

A *positive unit pivot* at (r, c) is permitted when the current entry is one. It removes row r and column c, replacing every surviving entry by

$$A'_{sh} = A_{sh} + A_{sc}A_{rh}, \qquad s \neq r,\ h \neq c. \tag{1}$$

This is an unsigned, fill-producing operation. It is not signed Gaussian elimination, a Schur complement with a minus sign, or an assertion that algebraic cancellation of a presentation is geometrically valid.

For an eliminable set $S \subseteq R$ put $w(S) = \sum_{r \in S} w(r)$. Define

$$\mathrm{OPT}(A, w) = \max\{w(S) : S \text{ is eliminable}\}, \qquad \mathrm{def}(A, w) = w(R) - \mathrm{OPT}(A, w).$$

Rows in $R \setminus S$ are passive during the selected eliminations. Their transformed entries are retained, even if a row becomes numerically zero.

Definition 2.1 (Source-private order)**.** A source-private order is a sequence of distinct rows and distinct columns $(r_1, c_1), \ldots, (r_k, c_k)$ such that

$$A_{r_i c_i} = 1, \qquad A_{r_j c_i} = 0 \quad \text{for all } j > i. \tag{2}$$

The entries in this definition are the *original* entries. Passive rows need not vanish in c_i.

Within the selected rows a source-private pivot produces no fill into a later selected row. Consequently Equation (2) remains valid throughout its execution. Passive rows may acquire large multiplicities; that separate issue is treated in Section 8.

2.2 Why positive fill cannot eliminate more rows

The next proposition is inherited from the inspected positive-cancellation work. We include the obstruction argument because it explains why the immutable source graph is sufficient.

Definition 2.2 (Stopping set)**.** A nonempty set Q of rows is a stopping set if every column has total multiplicity on Q either zero or at least two:

$$\sum_{r \in Q} A_{rc} \neq 1 \quad (c \in C).$$

Lemma 2.3 (Persistence of a stopping obstruction)**.** *Starting from a stopping set, arbitrary positive unit pivots cannot eliminate every row of that set.*

Proof. Suppose the current stopping set is Q and pivot (r, c) is performed. Write $s_h = \sum_{q \in Q} A_{qh}$. If $r \notin Q$, its new total on any surviving column h is

$$s'_h = s_h + s_c A_{rh}.$$

Both s_h and s_c are zero or at least two, so s'_h cannot equal one.

If $r \in Q$, then $s_c \geq 2$ because $A_{rc} = 1$. Thus $Q \setminus \{r\}$ is nonempty. On each surviving column,

$$s'_h = (s_h - A_{rh}) + (s_c - 1)A_{rh} = s_h + (s_c - 2)A_{rh}.$$

Again this is zero or at least two: if $s_h = 0$, then $A_{rh} = 0$; otherwise $s_h \geq 2$. The surviving rows of Q therefore remain a nonempty stopping set. Induction proves the claim. □

Proposition 2.4 (Positive/private equivalence)**.** *For unrestricted positive unit pivots, a set S of original rows is eliminable if and only if the submatrix on S has a complete source-private order. Hence* $\mathrm{OPT}(A, w)$ *may be optimized entirely in the original matrix.*

Proof. Repeatedly remove a column of total multiplicity one on the live subset of S, along with its unique incident row. If the process stops with nonempty rows, those rows are a stopping set. By lemma 2.3, even fill-producing pivots cannot remove all of them. If it empties S, the removal order satisfies Equation (2) and is a valid positive order. An order eliminating S in the presence of passive rows restricts to exactly the same selected-row operations, because passive rows never act as donors. This proves both directions. □

Sometimes only a specified subset $E_* \subseteq \{(r, c) : A_{rc} = 1\}$ is authorized for *source-private* pivots. The algorithm below supports this restriction. Without further hypotheses, we do not extend proposition 2.4 to arbitrary fill-producing pivots whose authorization changes during transport. Such an extension would be a different problem.

Proposition 2.5 (Safe full private prepeeling)**.** *If column c has exactly one live occurrence, at row r, and (r, c) is eligible, some optimum source-private plan contains r. Deleting this pair reduces the optimal value by exactly $w(r)$.*

Proof. For any feasible selected set omitting r, prepend (r, c). Its column is absent from every other live row, so it does not disturb their order. Since $w(r) \geq 0$, this cannot reduce the objective. If an optimum already contains r, its remaining selected subset is still source-private eliminable; prepend (r, c) to an order for that subset. Conversely every plan on the remaining rows can be preceded by (r, c). This gives equality of the two optima after adding $w(r)$. □

2.3 The pivot-capable support graph

A column is *pivot-capable* if it has at least one eligible unit entry. Let G be the bipartite graph on R and the pivot-capable columns, with an edge for *every* positive entry in those columns. Nonunit edges remain present as possible alternating-cycle obstructions. Columns without an eligible unit cannot be selected, so they are omitted only from this optimization graph.

This omission is not geometric deletion. Such columns still occur in passive transport and in the complete zero/one-handle graph. In particular, the forest routine must retain one-handles absent from G.

For a matching $M \subseteq E_*$, orient its edges from row to column and every other support edge from column to row. Call the resulting digraph D_M.

Lemma 2.6 (Alternation, acyclicity, and triangularity)**.** *For an eligible matching M, the following are equivalent:*

(i) M is uniquely restricted: its saturated vertices induce a graph having M as its unique perfect matching.

(ii) No simple cycle alternates between edges in M and edges outside M.

(iii) D_M is acyclic.

(iv) The pairs in M admit a source-private order.

Proof. Two distinct perfect matchings on the same vertex set have a symmetric difference containing an alternating cycle. Conversely, reversing membership around an alternating cycle gives a different perfect matching. This proves (i)⇔(ii).

At a row, the only possible outgoing edge of D_M is its matched edge; at a column, the only possible incoming edge is its matched edge. Every directed cycle therefore alternates. An alternating cycle has one of its two traversal directions directed. This proves (ii)⇔(iii).

Contract each matched pair to its index. An entry $A_{r_i c_j} > 0$ with $i \neq j$ gives a directed dependency from pair j to pair i in D_M. If the dependencies are acyclic, a reverse topological ordering makes every such i precede j. Exactly this condition says $A_{r_j c_i} = 0$ for later j. A source-private ordering, in turn, forbids a directed dependency cycle. Vertices unmatched by M cannot lie on a directed cycle and cause no extra condition. This proves (iii)⇔(iv). □

Assign edge (r, c) weight $w(r)$. Since a matching uses a row at most once, maximizing its edge weight maximizes cancelled row weight. Thus our task is an eligible, row-weighted uniquely restricted matching problem. The equivalence is classical in spirit [3]; the eligibility and source interpretation are important for this application.

3 Exact composition over biconnected blocks

A *block* means an edge block in the vertex-biconnected decomposition: either a maximal biconnected subgraph containing a cycle or a single bridge. Isolated vertices have no edge block. The graph whose nodes are blocks and their incident original vertices is a forest. Keeping nonarticulation vertices as leaves is convenient for a uniform implementation.

For a connected block B define its cycle rank

$$b(B) = |E(B)| - |V(B)| + 1.$$

A bridge has rank zero. We set $\kappa = 0$ for a forest and otherwise take the maximum rank of its cyclic blocks. This is not the graph's total cycle rank.

Lemma 3.1 (Locality of alternating-cycle constraints)**.** *A union of matchings chosen within the blocks is a uniquely restricted matching in G if and only if it uses at most one incident selected edge at each articulation vertex and is uniquely restricted inside every block.*

Proof. The capacity condition is exactly what makes the union a matching. Every simple undirected cycle belongs to one biconnected block. Therefore every alternating cycle belongs to one block. Apply lemma 2.6. The converse follows by restriction of any global matching and any global alternating cycle. □

Root each block-incidence tree at an original vertex. A block with parent vertex v sends two response scores:

$$m_B(0),\ m_B(1),$$

according as v is unmatched or matched by an edge of B. The scores include all edges in the block and all descendant subtrees but not the parent side. An impossible state has score $-\infty$.

At an original vertex v with child blocks $B_1, \dots, B_t$, define a_v as the best descendant score when the parent block occupies v, and f_v as the best score when the parent does not occupy v. Then

$$a_v = \sum_{i=1}^{t} m_{B_i}(0), \tag{3}$$

$$f_v = a_v + \max\left(0, \max_{1 \le i \le t} \{m_{B_i}(1) - m_{B_i}(0)\}\right). \tag{4}$$

The empty maximum contributes zero. State zero is always feasible, since the block can choose no incident edge at its parent.

Proposition 3.2 (One-bit articulation interface)**.** *Equations Equations* (3) *and* (4)*, together with exact local block responses, compute the global optimum. No ordering or reachability relation between different child blocks is needed.*

Proof. If the parent occupies v, all child blocks must leave it unmatched. Otherwise at most one child can occupy it. These are exactly the alternatives in the equations. Edge weights belong to their unique block, so no selected row weight is counted twice. By lemma 3.1, no alternating cycle can combine two child blocks across their only common vertex. Induct up the block-incidence tree and use f_v at each root. □

The bit records *capacity use*, not global saturation by an unspecified future edge. Confusing these two meanings would double-count solutions at joins. The implementation stores witnesses as shared expression nodes rather than copying an entire partial matching into every response.

4 Six-state compression of degree-two paths

4.1 Boundary information that a path actually exposes

Let

$$P = (v_0, e_1, v_1, e_2, \ldots, e_\ell, v_\ell)$$

be a path in a block whose internal vertices all have degree two *within that block*. An internal vertex may have many child blocks in the original graph; those are already summarized by (a_v, f_v). Endpoints are terminals retained in the block skeleton. They may coincide when the whole block is a cycle opened at its parent.

For a matching on P, let $s_i \in \{0, 1\}$ indicate whether e_i is selected. We require $s_i + s_{i+1} \leq 1$, and a selected edge must be eligible. Orient each path edge as in D_M. Its orientation relative to the listed path order is $o_i \in \{+, -\}$.

The path exposes

$$(\alpha, \beta, d), \qquad \alpha = s_1, \quad \beta = s_\ell, \quad d = \begin{cases} + & \text{all } o_i = +, \\ - & \text{all } o_i = -, \\ 0 & \text{otherwise.} \end{cases} \tag{5}$$

Thus d records whether the path is a directed connection between its endpoints, and in which direction. The score includes selected-edge weights and the descendant contribution of each internal vertex: a_v if either adjacent path edge is selected, and f_v if neither is selected. Endpoint descendant scores are excluded.

Lemma 4.1 (Six-state lemma)**.** *For a fixed bipartite path with fixed endpoint types, at most six triples in Equation (5) are feasible. The exact best score and one realizing matching for every feasible triple can be computed with $O(\ell)$ arithmetic operations.*

Proof. There are at most four triples having $d = 0$, one for each endpoint-flag pair. When $d = +$, the orientation required of each edge uniquely determines whether it is selected: a row-to-column edge is selected and a column-to-row edge is not. In particular both endpoint flags are forced. The same holds for $d = -$. Eligibility or matching constraints may remove these possibilities but cannot create more. Hence there are at most $4 + 1 + 1 = 6$ feasible states.

For the dynamic programme, fix the first-edge flag α. After processing i edges retain only the last flag s_i, the common direction d_i (zero once two orientations have differed), and the best score for that state. On choosing the next flag s, reject $s_i + s > 1$ and any ineligible selection. At internal vertex v_i, add a_{v_i} if $s_i + s = 1$, otherwise add f_{v_i}. Add $s\, w(e_{i+1})$. The new direction is

$$d_{i+1} = \begin{cases} d_i & d_i = o_{i+1}, \\ 0 & d_i \neq o_{i+1}. \end{cases}$$

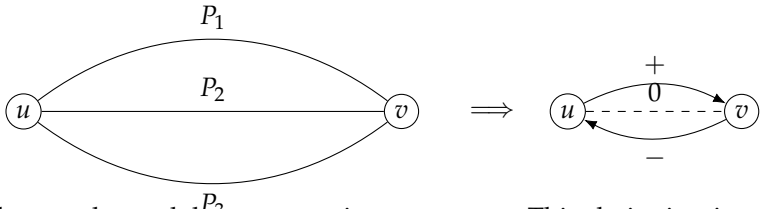
Once zero, it remains zero. There are constantly many states and two transition choices per state. Initialize from e_1 and repeat. The usual induction on the processed prefix proves optimality of each retained score. Backpointers recover a witness without retaining an exponential number of prefixes. □

Lemma 4.2 (Exact replacement inside a block)**.** *Replace each internally disjoint degree-two path by its six-state table. After choosing one state per path, impose endpoint capacity at most one, and put a directed*

skeleton arc precisely for states with $d \neq 0$. The original block matching is uniquely restricted if and only if this directed skeleton is acyclic.

Proof. Every internal vertex has only its two path edges in the block. A simple directed cycle entering the internal portion of a path must traverse the entire path to an endpoint; it cannot turn around without repeating an edge or vertex. Consequently a directed cycle in the original block contracts to a directed closed walk in the skeleton, and such a walk contains a directed cycle.

Conversely, replace the arcs of a simple directed skeleton cycle by their directed paths. Distinct paths have disjoint interiors, so the result is a directed cycle in the original block. A self-loop arc is precisely a directed cycle through a whole cycle block. Capacity checks at the endpoints, together with the internal checks in lemma 4.1, ensure that the union is a matching. Now apply lemma 2.6. □



Three paths; each has at most six states. Only directed through-paths become arcs.

This choice is rejected: the skeleton has a directed cycle.

Figure 1: The acyclicity test is on a branching skeleton, not on an expanded family of path vertices. Endpoint matching flags must also be checked.

4.2 Special cases that must not be dropped

A length-one path has no internal vertex contribution. Its endpoints cannot both be occupied by different edges of that same path: selecting the unique edge occupies both endpoints and adds its weight once. The six-state construction handles this without a special combinatorial assumption.

When $v_0 = v_\ell$, both flags consume capacity at the *same* root. Thus the test is $\alpha + \beta \leq 1$, not two unrelated endpoint tests. Moreover, either nonzero direction gives a self-loop and is forbidden. This is the mechanism that excludes perfect matchings on a cycle.

A nonunit edge is never selected, but its unmatched orientation is still retained. Dropping such edges would incorrectly destroy alternating-cycle obstructions. Similarly, path endpoints must retain their row/column types; an untyped direction convention can reverse dependencies after a path is concatenated.

5 The cycle-block kernel theorem

5.1 A small branching skeleton

Lemma 5.1 (Path count from block cycle rank)**.** *Let B be a cyclic biconnected block with designated parent vertex r. If $b = b(B) \geq 2$, retain r and every vertex of degree at least three in B, then suppress all other degree-two vertices into paths. The resulting multigraph has at most $2b - 1$ terminals and at most $3b - 2$ paths. If $b = 1$, B is a cycle and is represented by one root-to-root path.*

Proof. A cyclic biconnected block has minimum degree two. Since $|E| = |V| + b - 1$,

$$\sum_{v \in V(B)} (\deg_B(v) - 2) = 2b - 2.$$

Every degree-at-least-three vertex contributes at least one, so there are at most $2b - 2$ such vertices. Adding r when its degree is two yields at most $h = 2b - 1$ terminals.

For $b \geq 2$ some vertex has degree at least three. Every maximal chain of nonterminal degree-two vertices ends at terminals; otherwise it would be a separate cycle component. Suppression preserves connectedness and cycle rank. If p is the number of resulting paths, then $p - h + 1 = b$, giving $p \leq 3b - 2$.

For $b = 1$, the degree identity forces degree exactly two at every vertex. The connected block is a cycle. Keep r and open the cycle there into a single path whose two endpoints are r. □

Theorem 5.2 (Exact cycle-block optimization)**.** *Let A be a finite nonnegative integer matrix with nonnegative integer row weights and a specified set of eligible original unit pairs. Let G be the pivot-capable support graph, let $n = |V(G)|$, $m = |E(G)|$, and let m_0 count all nonzero entries of A, including columns omitted from G. Set*

$$p_b = \begin{cases} 1 & b = 1, \\ 3b - 2 & b \geq 2. \end{cases}$$

There is an exact algorithm producing an optimal eligible source-private plan with arithmetic work

$$O\left(n + m_0 + \sum_{B \text{ cyclic}} (b(B) + 1) 6^{p_{b(B)}} \right). \tag{6}$$

Deterministic sorting of external labels may add $O(\mathsf{S} \log(\mathsf{S} + 2))$ comparisons. In binary bit complexity the bound is

$$\mathsf{S}^{O(1)} 2^{O(\kappa)}, \qquad \kappa = \max_B b(B), \tag{7}$$

where S includes the original sparse matrix, labels, weights, and eligibility data. When all original unit pairs are eligible, this also optimizes unrestricted positive unit cancellation.

Proof. Compute the block decomposition and root its incidence forest. Handle a bridge directly by its two choices: select it if eligible or leave it unselected. For a cyclic block, apply lemma 5.1 and compute every path table by lemma 4.1. Internal descendant potentials have already been obtained from lower blocks.

Enumerate at most 6^{p_b} combinations of path states. Reject combinations violating terminal capacity. For the others, test acyclicity of the directed skeleton, using lemma 4.2. Its vertex and edge counts are $O(b + 1)$, so the check costs $O(b + 1)$. Add the appropriate descendant potential at each terminal other than the parent, and keep the best score for each parent-capacity flag. Completeness follows because every feasible block matching induces an enumerated combination. Soundness follows because the realizing path matchings have disjoint interiors, compatible endpoint capacities, and an acyclic skeleton. Therefore the block responses are exact.

Apply proposition 3.2 to combine them. Block edges partition $E(G)$; total block-incidence size is $O(n + m)$. Thus all path processing, bridge handling, and articulation combinations have linear graph/arithmetic work, apart from skeleton enumeration. Source admission and graph construction additionally inspect the m_0 original nonzeros, including omitted columns. Explicitly stored zero entries, if present, are charged separately to input admission. This gives Equation (6).

Each score is a sum of weights of a subset of the original rows. If input weights use at most B_w bits, every score uses $O(B_w + \log(|R| + 1))$ bits. Arithmetic operations on these scores, original counts, and labels therefore have polynomial bit cost. There are at most m cyclic blocks and $p_b = O(b)$, yielding Equation (7). Witness extraction and independent private-order replay are polynomial in the explicit input and output size. Finally apply proposition 2.4 in the unrestricted case. □

Corollary 5.3 (Many easy blocks remain easy)**.** *If every block of G has cycle rank at most a fixed constant, weighted eligible source-private optimization has polynomial bit complexity and linear graph/arithmetic work. In particular this holds for cactus graphs, even with an unbounded number of cycles and an unbounded degree at articulation columns.*

The cactus consequence is a compatible weighted/eligible formulation of a classical tractable class, not a new discovery that cacti admit efficient uniquely restricted matching [3]. The new useful parameter statement is the explicit six-state dependence on *individual* block ranks and its binary repetition interface.

5.2 Preprocessing and witness accounting

The private queue of proposition 2.5 can run first. Removing rows and deleting now-incapable columns cannot increase cycle rank of any surviving cyclic block: each surviving cycle lies in an original block, and edge deletion cannot increase the dimension of its cycle space. Consequently the parameter measured after prepeeling is no larger than before, and may be much smaller.

Our implementation records maximizing witnesses in a persistent expression arena. A join records references to child witnesses; selecting an edge records its source pair. Flattening is iterative. This avoids repeatedly copying a long partial matching at every node of the block tree. The final explicit schedule still has at least one record per selected row, as it must.

A general plan replay verifies feasibility, source labels, unit entries, distinct pivots, private ordering, row inventory, and the reported weight totals. It does *not* independently prove the optimizer's global optimality. The latter rests on the complete algorithm and its proof. The binary-power certificate in the next section has a stronger, separate all-state optimality replay contract.

6 Binary repetition and complete six-state certificates

6.1 A typed transition algebra

The path dynamic programme has a fixed six-element working state space

$$Q = \{0, 1\} \times \{-, 0, +\}.$$

The first coordinate remembers the previously selected edge; the second remembers whether the processed prefix remains directed throughout. The first endpoint flag is kept outside this working state and tried in both possible ways.

For a transition adding the next edge, the local data are: the type of its left vertex (row or column), whether the edge is eligible, its selected-edge weight, and the occupied/free descendant

scores at that vertex. Let T be the 6×6 matrix whose entry $T_{qq'}$ is the best added score for a legal transition from q to q', or $-\infty$ when no such transition exists. Define max-plus multiplication by

$$(U \otimes V)_{ij} = \max_{k \in Q} (U_{ik} + V_{kj}). \tag{8}$$

Its identity has diagonal zero and off-diagonal $-\infty$. Splitting a transition sequence at an intermediate state proves associativity directly: both parenthesizations maximize the same sum over the same intermediate states.

For a supplied alternating template, consecutive transitions alternate between two matrices T_A, T_B. After the first edge of a length-ℓ path, write $\ell - 1 = 2q + \epsilon$, where $\epsilon \in \{0, 1\}$. The remaining transition product is

$$(T_A \otimes T_B)^{\otimes q} \otimes T_A^{\otimes \epsilon}. \tag{9}$$

The matrices are typed: A and B refer to specific row/column parity, edge eligibility, and internal-vertex potentials. The first edge is initialized separately so it does not receive an internal-vertex contribution at the left endpoint.

Definition 6.1 (Repeat-safe path template)**.** A repeat-safe path template specifies alternating row/column types, eligible-edge flags, selected-edge weights, and internal occupied/free potentials. Every repeated internal vertex and every represented descendant context is fresh; only consecutive path endpoints are identified. The final two endpoints and their contribution ownership are declared explicitly. A cycle closure is a separate operation on the final table.

Freshness matters. Repeating a label for the *same* vertex would introduce an additional capacity constraint, and repeating a shared descendant would count its weight multiple times. Such repetitions are not authorized by a short string that merely resembles an alternating pattern.

Theorem 6.2 (Binary path powers)**.** *For a supplied repeat-safe alternating path of binary length $\ell \geq 1$, its exact six-state response table is computable with $O(\log(\ell + 1))$ fixed-size max-plus matrix products. If local scores have at most B bits, all finite table entries have $O(B + \log(\ell + 1))$ bits. Construction and complete-table certification therefore have polynomial bit complexity in $B + \log(\ell + 1)$.*

Proof. Use binary exponentiation in Equation (9). By induction on concatenation, matrix entries are precisely the optimal scores over the represented transition words. Initializing both possible first-edge choices and reading the final previous-edge/direction state yields the triples of Equation (5). This proves exactness.

Each transition contributes a nonnegative sum of a bounded number of local scores, each at most 2^B. A path score is at most $O(\ell 2^B)$, so finite entries need $O(B + \log(\ell + 1))$ bits. A matrix product has only 6^3 candidate sums and comparisons. Binary exponentiation uses $O(\log(\ell + 1))$ products, proving the claimed encoded cost. □

6.2 Certifying the maximum, not only a realizing path

A proof packet stores leaf transition matrices, an identity leaf, and a directed acyclic graph of product nodes. A node gives references to earlier nodes and its claimed full 6×6 matrix. The output references the final node and its extracted path table.

An independent verifier reconstructs every leaf from the supplied local template. At each product node it recomputes all candidate sums in Equation (8), checks equality to the claimed matrix, and verifies the represented transition-word length and alternating type. Checking only the maximizing witness would establish a lower bound, not an optimum; all candidates are essential.

Proposition 6.3 (Complete power-table replay)**.** *Acceptance by the described verifier proves that every output table entry is the actual optimum for the declared repeat-safe path template. It also proves infeasibility of every omitted state. The certificate has* $O(\log(\ell+1))$ *matrix nodes and*

$$O((B+\log(\ell+1))\log(\ell+1))$$

bits of scalar matrix data, up to reference and serialization overhead.

Proof. Leaf correctness is checked from the transition definition. Inductively, exact equality to every entry of the max-plus product proves both the upper bound over all concatenated runs and attainability by some compatible pair of runs. The word check prevents using a product with the correct-looking numerical value but the wrong repetition count or parity. Final extraction checks both endpoint flags and direction. The bit estimate follows from Theorem 6.2 and the node count. □

This is an arithmetic/template certificate. It does not certify that a three-dimensional source admits the supplied template. That requires a source exporter proving both incidence equality and repeat freshness.

Corollary 6.4 (A binary cycle)**.** *For a simple all-unit bipartite cycle of* $2N$ *edges with all row weights equal to* $w \geq 0$*, where* $N \geq 2$*,*

$$\mathrm{OPT} = (N-1)w, \qquad \mathrm{def} = w.$$

These values and the complete path table obtained by opening the cycle at a root have certificates polynomial in the binary length of N *and* w*.*

Proof. A matching of size N is perfect, and the entire cycle alternates, so it is not uniquely restricted. A matching of size $N-1$ exists and cannot contain an alternating cycle: the graph's only simple cycle uses all its vertices. This proves the formula, including $w = 0$.

Alternatively open the cycle at its root, use Theorem 6.2, and retain exactly the states with $\alpha + \beta \leq 1$ and $d = 0$. These are precisely the legal cycle states from lemma 4.2. The certificate checks the whole table before this filter is applied. □

6.3 What a binary summary does not output

For $N = 2^{1024}$, the cycle has 2^{1025} edges. A flat matching or pivot list of size $N-1$ cannot be written in time polynomial in $\log N$. The implemented binary routine returns an exact symbolic score and full table proof, *not* that flat list. The explicit matrix optimizer returns an explicit order only for an explicit matrix.

The six-state tables can also be used on a *supplied* small branching skeleton whose edges represent repeat-safe paths. The same block proof computes the optimum score from these tables without expanding the paths. A general compressed source-private trace, its geometric execution, and the transport of exponentially many distinct passive row identities would require additional output conventions. No such general compiler is asserted here.

7 Weighted partial cancellation without primal-forest search

The source handle structure has a graph H whose vertices are zero-handles and whose edges are one-handles. Loops and parallel edges are allowed. Its complete edge inventory must be retained even when some one-handle occurs in no two-handle attaching curve.

For this section, A contains *uncapped* occurrence counts in H. We assume every row reduces modulo two to a cycle:

$$\partial_H(A_{r,\cdot} \bmod 2) = 0 \quad \text{in } \mathbb{F}_2^{V(H)}. \tag{10}$$

A closed attaching walk implies this identity. The identity alone is sufficient for the following graph theorem, but it does not authenticate embedded attaching curves.

Lemma 7.1 (Private pivot edges are cographically independent)**.** *Let $(r_i, c_i)_{i=1}^k$ be a source-private order satisfying Equation (10). For every i, the edge c_i is not a bridge of*

$$H - \{c_1, \dots, c_{i-1}\}.$$

Consequently deleting all pivot edges preserves every connected component of H.

Proof. The original row r_i has zero entries at earlier pivot columns by Equation (2). Thus its parity cycle survives in the graph with those columns deleted. It uses c_i with odd multiplicity, indeed multiplicity one. Every mod-two cycle meets each cut in an even number of edges, counted modulo two. A bridge alone forms a cut, so it cannot belong to this cycle. Hence c_i is not a bridge. Delete the pivot edges in order to obtain the component claim. □

Theorem 7.2 (Weighted partial deferred forests)**.** *Under Equation (10), every source-private partial plan admits a spanning forest of H disjoint from its pivot edges. Any already prescribed forest disjoint from those pivots extends to such a spanning forest.*

Let $A \setminus F$ denote the matrix obtained by omitting the columns of a spanning forest F of H. Then

$$\max_{F \text{ spanning forest of } H} \mathrm{OPT}_{\mathrm{private}}(A \setminus F, w) = \mathrm{OPT}_{\mathrm{private}}(A, w). \tag{11}$$

Thus one may optimize the weighted partial source-private plan first and choose a forest afterwards. In the unrestricted positive calculus, proposition 2.4 identifies these values with the corresponding positive-cancellation optima.

Proof. By lemma 7.1, H minus the pivot edges has the same components as H, so it has a spanning forest of each original component. Any prescribed acyclic set in this complement can be extended by adding edges between its distinct components until the forest spans.

A source-private order found in $A \setminus F$ is still source-private in A: restoring other columns does not change its selected unit entries or the zero entries in later rows at its pivot columns. Therefore the left side of Equation (11) is at most the right side.

Conversely take an optimal order in A and choose a spanning forest disjoint from its pivots. Omitting forest columns preserves that order and its weight. Hence its weight is attained on the left side. This gives equality. □

The full-elimination form of this argument already appears in report 105 [7]. The present statement is the weighted partial consequence needed to compose that principle with Theorem 5.2; its proof also records the prescribed-forest extension. It is not a new claim that arbitrary geometric handle moves commute with every forest contraction.

7.1 Why saturation is insufficient for this contract

Counts 0, 1, and "at least 2" suffice to decide whether a pair is a private unit. They do not suffice for Equation (10): replacing three by two changes parity. Accordingly the delivered adapter counts every original one-handle occurrence before forming a saturated optimization view. The forest routine tests the uncapped row parity on the complete graph.

If a caller already has a preferred forest that intersects an optimal plan's pivots, Theorem 7.2 does not preserve that forest. One must either forbid those pivot columns before optimization, or allow the forest to change. Likewise, optimizing a secondary geometric cost of the forest is not solved by the arbitrary extension routine.

7.2 A concrete forest-choice witness

The audit includes a graph $H = K_4$ with edge columns

$$0 = 01, \quad 1 = 02, \quad 2 = 03, \quad 3 = 12, \quad 4 = 13, \quad 5 = 23$$

and rows

$$A = \begin{pmatrix} 0 & 0 & 0 & 2 & 0 & 0 \\ 0 & 0 & 2 & 0 & 2 & 0 \\ 1 & 1 & 0 & 0 & 3 & 1 \\ 1 & 1 & 2 & 1 & 2 & 0 \end{pmatrix}, \qquad w = (3, 8, 1, 3).$$

Every row satisfies uncapped cycle parity. The plan $(2, 5), (3, 1)$ has weight four and leaves weight eleven. It is compatible with the forest $\{0, 2, 3\}$. Among the sixteen spanning trees, the optimum is three for some fixed choices, including $\{0, 1, 5\}$, and four for the best choices. The first two rows have no unit entries and cannot be removed; this proves the upper bound four without trusting enumeration.

This example is abstract closed-walk graph data, not a claimed knot-exterior realization. Its full matrix, the plan, the chosen forest, and all sixteen exact forest scores are saved in `examples/forest_choice_witness.json`.

8 Linear-size positive transport circuits

8.1 An immutable triangular dependence structure

Fix a source-private order $(r_i, c_i)_{i=1}^{k}$. Write $D = C \setminus \{c_1, \dots, c_k\}$ for the surviving columns and put

$$R_i = (A_{r_i h})_{h \in D}, \qquad N_{ij} = A_{r_i c_j} \quad (i < j).$$

The selected rows have zeros at all earlier pivot columns. Therefore earlier positive pivots never alter a later donor row. Only passive rows acquire fill during execution.

Define vector-valued expressions backwards by

$$T_i = R_i + \sum_{j > i} N_{ij} T_j \qquad (i = k, k-1, \dots, 1). \tag{12}$$

For a passive original row p, define

$$P_p = (A_{ph})_{h \in D} + \sum_{i=1}^{k} A_{p c_i} T_i. \tag{13}$$

Every coefficient in these expressions is an original input coefficient. References go only to later donor indices. The expressions are a directed acyclic circuit of vector sums and nonnegative scalar multiples.

Theorem 8.1 (Exact fill-free numerical transport)**.** *The vectors P_p of Equation (13) are exactly the surviving rows obtained by executing the positive pivots in the specified order. Their circuit uses $O(\mathsf{S})$ input-coefficient occurrences and references. For any integer linear observable $\lambda : \mathbb{Z}^D \to \mathbb{Z}$ specified on the coordinate vectors, all values $\lambda(P_p)$ can be computed with $O(\mathsf{S})$ arithmetic operations without expanding the residual matrix.*

Proof. The final effect of one occurrence of pivot c_i is its donor outside part R_i, plus its original occurrences of later pivots replaced by their own final effects. Earlier pivots do not occur in the donor and never alter it. Backwards induction therefore gives precisely Equation (12). A passive row initially has its outside part and a specified number of occurrences of each pivot. Positive substitution is linear in these occurrences, yielding Equation (13). This is the same recursion obtained by following all forward fill operations, but its shared donor effects are not duplicated.

Each original row entry appears once as an outside coefficient, a later-donor reference, a passive reference, or the pivot unit that is omitted. Thus the representation is linear in the sparse input description. Apply λ to each R_i, then evaluate donor values in descending order using Equation (12), and finally evaluate every passive expression. Every stored coefficient is used a bounded number of times. This proves the operation bound. □

In matrix notation, if N is strictly upper triangular, then

$$T = (I - N)^{-1}R = (I + N + \cdots + N^{k-1})R.$$

The minus sign here describes the solution of $T = R + NT$; it does *not* change the positive pivot rule into signed elimination. Implementing the circuit does not construct this inverse or any dense power.

Proposition 8.2 (Transport bit costs)**.** *Suppose nonzero input counts have at most B bits and observable coefficients have absolute value at most 2^D. Let m_0 be the number of input nonzero entries. Every integer used in the scalar circuit evaluation has*

$$O\big(D + (k+2)(B + \log(m_0 + 2))\big)$$

bits. Consequently Theorem 8.1 is a polynomial bit-complexity algorithm, not merely a unit-cost arithmetic statement.

Proof. Expanding the acyclic recursion conceptually, a coefficient is a sum over directed donor paths. Such a path visits each donor at most once. A term uses at most $k+1$ input factors and one observable coefficient, and therefore has magnitude at most $2^{(k+1)B+D}$. A coarse bound on the number of terms is $(m_0 + 2)^{k+2}$. Taking the binary logarithm gives the stated estimate. This bounds intermediate partial sums as well; with signed observables, use the sum of absolute term values. Exact integer additions and multiplications at this bit length are polynomial-time. □

8.2 A genuine output-size separation

Proposition 8.3 (Quadratic passive fill at block rank zero)**.** *For arbitrary $p, s \geq 1$, there is an input with $p + s + 1$ nonzero entries whose pivot-capable graph is a tree, and a one-pivot optimal weighted plan whose explicit residual has ps nonzero entries. Its circuit still has $O(p + s)$ size.*

Proof. Use a donor row r with $A_{rc} = 1$ and $A_{rd_j} = 2$ for $1 \le j \le s$. Use p further rows q_i, each having the single entry $A_{q_i c} = 1$. Give r weight $p + 1$ and each q_i weight one. Only column c is pivot-capable, so the optimization graph is a star.

At most one row can be matched, since only c can be a pivot. The donor is the unique highest-weight choice. Pivoting (r, c) places the entry two in every position (q_i, d_j), giving ps nonzeros. The circuit stores the donor's s outside entries and one reference from each passive row. This has $O(p + s)$ size. The observable summing outside entries has value $2s$ on every passive row and can be evaluated without writing ps entries. □

A residual matrix API that insists on explicit entries therefore has an unavoidable quadratic output requirement on this family. A query API should retain the circuit when it needs totals, signed linear probes, or a small number of marked linear observables. Repeatedly asking all individual coordinates recreates the output-size cost and is not advertised as linear overall.

8.3 Numerical transport is not attaching-curve transport

The circuit remembers multiplicity and source row identity, not the cyclic order of passages, ribbon data, the placement of bands, or protected boundary patterns. Its structural validator checks that references are well formed; it does not authenticate a circuit supplied by an untrusted source. The constructor binds its coefficients to a separately validated source-private plan and original matrix. Neither layer replaces an independent geometric cancellation verifier.

In particular an empty passive vector is retained with its original row label. It is not a proof that the corresponding two-handle disappears. The resulting status remains a numerical reduction plan until the geometric layer has transported and checked the actual attaching maps.

9 Residual barriers and comparison with other parameters

9.1 Easy optimization can leave an arbitrarily large core

Theorem 9.1 (Connected cactus deferral barrier)**.** *For $t \ge 1$, take rows x_i, y_i and columns c_0, c_i for $1 \le i \le t$, with*

$$\operatorname{supp} A_{x_i,\cdot} = \operatorname{supp} A_{y_i,\cdot} = \{c_0, c_i\}, \qquad A_{x_i c_0} = A_{x_i c_i} = A_{y_i c_0} = A_{y_i c_i} = 1.$$

Give the pair weights $a_i, b_i \ge 0$. The support graph is connected, every cyclic block has rank one, and

$$\mathrm{OPT} = \sum_{i=1}^{t} \max(a_i, b_i), \qquad \mathrm{def} = \sum_{i=1}^{t} \min(a_i, b_i). \tag{14}$$

In particular, unit weights force at least t unresolved rows despite $\kappa = 1$.

Proof. Each pair $\{x_i, y_i\}$ is a stopping set: its two supported columns both have total two. By lemma 2.3, a cancellable selected set cannot contain both rows of a pair. Its weight is therefore at most the first sum in Equation (14).

Select a heavier row from each pair and match it to its private column c_i. Each such column has exactly one occurrence among the selected rows, so all these rows have a source-private order. This realizes the upper bound. The total row weight gives the deferral formula. The graph consists of t four-cycles meeting only at c_0, so those cycles are precisely its cyclic blocks. □

The shared column has degree $2t$. Thus this family is not confined to a regime in which every active column meets at most two rows. It demonstrates why block decomposition is useful even when an articulation has a large interface degree. Nevertheless, it also proves that a bound on block rank cannot alone bound the number or weight of remaining two-handles.

For actual source geometry, a positive deferral value is only an obstruction to full elimination in this fixed raw calculus. It does not prove knottedness, non-handlebody status, nonball status, or failure after other geometric slides or changes of charts. Conversely, a low optimum deferral is not a complete geometric certificate either.

9.2 The parameter is local but not universally superior

The total cycle rank can be arbitrarily larger than κ: the family of Theorem 9.1 has total rank t and maximum block rank one. A global enumeration over all feedback edges would fail to exploit this distinction.

On the other hand, $K_{2,q}$ is one biconnected block for $q \geq 2$, with cycle rank $q-1$ and treewidth two. Its treewidth is at most two by the bags consisting of the two vertices on the small side and one vertex on the other side; it is at least two because it contains a cycle. Thus a bounded-width method may be dramatically preferable to our block-rank enumeration.

For perspective, selecting one endpoint from each of b non-tree edges of a connected block leaves a forest after those selected vertices are deleted. Adding these at most b vertices to every bag of a forest decomposition gives treewidth at most $b+1$. Hence the inspected $2^{O(w\log(w+2))}$ ordered-interface bound can be specialized to $2^{O(\kappa\log(\kappa+2))}$ using suitable blockwise handling [9, 10]. Our direct six-state reduction removes that extra logarithm in this *particular parameter*, while it may lose badly when width is much smaller than cycle rank.

No lower bound shows the factor 6^{3b-2} to be optimal. It is a transparent upper bound before capacity pruning. Many state combinations are immediately incompatible. The code counts only combinations that survive its incremental capacity checks before running the acyclicity test. These empirical counts are not a replacement for the worst-case bound.

9.3 A safe local portfolio

An implementation should retain inexpensive complete-success paths. Run forced private peeling first. If the residual is empty, no matching optimizer is necessary. Otherwise compute block ranks, which costs linear graph work. Small-rank blocks can use this kernel; blocks with small validated decomposition width can use ordered interfaces; special incidence regimes can retain their more specialized solvers.

Proposition 9.2 (Budgeted incumbent prefix)**.** *Suppose a complete exact local fallback has bound $F(\mathsf{S})$ and an incumbent prefix is stopped after at most $P(\mathsf{S})$ charged work, where P is polynomial. Running the prefix first and, when necessary, recomputing with the fallback preserves a bound $P(\mathsf{S})+F(\mathsf{S})+\mathsf{S}^{O(1)}$. Reusing a sound completed answer is optional, not required for the bound.*

Proof. Charge all prefix work, including unsuccessful candidate construction and certificate replay, to P. On fallback, charge its complete run to F. Any conversion of an explicit source matrix, comparison of objectives, or replay of a returned plan has polynomial cost. There is at most one such prefix and one fallback in the stated procedure. □

A work limit is not a negative certificate. Nor is a per-call limit a bound on an unbounded number of calls. A full recognition dispatcher must charge retries and rebuilds globally; proposition 9.2 does not authorize repeatedly spending the same polynomial allowance without accounting for the number of repetitions.

10 Integration contract and a conditional route to quasi-polynomial cost

10.1 The maintained source boundary

The inspected native core interface exposes `handles0`, `handles1`, and `handles2`. A one-handle has a source identifier and two endpoint zero-handles. A two-handle carries a cycle of flags whose one-dimensional flags identify one-handles [5]. The delivered additive adapter reads this shape and computes *uncapped* counts. It then invokes the matrix optimizer and the parity-based deferred forest routine.

Its successful status is deliberately

`MATRIX_PLAN_NOT_GEOMETRY.`

The source-authentication and geometric-execution flags both remain false. The two adapter tests exercise this dictionary shape and parity rejection on abstract inputs. They are not source-geometric tests.

Before promotion, a native caller must reconstruct the counts from authenticated charts, not accept producer-declared multiplicities. It must replay every chosen pair through the actual geometric cancellation rules, preserving all passive attaching curves and protected data. It must keep one-handles omitted from the pivot graph, unresolved zero rows, and source row identities. A final handlebody or boundary-pattern census requires its own valid theorem even when the matrix plan is optimal.

The packet contains no patch that silently changes a maintained verdict path. Intake should place the report under the next available `Topology/UnknotRecognition/reports/` location and conduct native integration as separate work, consistent with the repository procedure [11].

10.2 Executed implementation and configured resource limits

The code uses Python's standard library. Source labels and numeric entries are checked as nonnegative integers, rejecting booleans. Graph decomposition and witness flattening are iterative. An explicit schedule is independently reconstructed from the selected matching and replayed against the source matrix.

The default optimizer limits include two million charged events, maximum block rank twelve, maximum path count 128 in a block, and one million admitted nonzero entries. These defaults are operational guards, not mathematical restrictions of Theorem 5.2. A completed invocation returns an exact optimum. Exceeding a configured allowance returns `RESOURCE_LIMIT`, which is inconclusive.

The current skeleton enumeration uses a bounded recursive routine; its configurable path guard is itself capped at 256 to avoid unbounded interpreter recursion. Thus this particular reference program is not advertised as an unrestricted solver for every finite graph regardless of configuration. The unrestricted mathematical algorithm uses the same finite enumeration with an ordinary explicit

stack. This distinction has no effect on completed cases or their proofs, but matters for truthful implementation claims.

The callback is invoked at charged stages and exceptions propagate. The counter is not a hard wall-clock or memory limit: sorting, Python allocations, serialization, and the independent replay are not all micro-instrumented. The CLI has a 32 MiB input admission check, but is not a hardened network service. The passive expansion guard checks cumulative produced dictionary sizes, including intermediate donor vectors; it is also not a strict preallocation memory ceiling.

10.3 The actual global proof obligation

Let n denote the original knot-diagram crossing count, or another explicitly fixed source encoding size. A local input may have many more represented cells than its encoded size. For a full algorithm, every transition from geometric state to matrix, every source proof, and every successor description must be charged to the chosen model.

Theorem 10.1 (Conditional hierarchy accounting)**.** *Suppose a sound and complete geometric recognition procedure has the following properties on inputs of size n:*

(i) it makes at most $Q(n)$ local optimizer calls, counting failed calls and repeats;

(ii) each explicit local matrix and its required provenance have encoded size at most $S(n)$, and maximum block rank at most $K(n)$ after safe preprocessing;

(iii) all geometric source construction, cancellation replay, residual repair, marked transport, terminal verification, and other work have total cost at most $H(n)$;

(iv) no expanded output omitted from these bounds is required; any use of binary path tables has a supplied repeat-safe exporter and an appropriate symbolic-output contract.

Replacing the local optimization stage by Theorem 5.2 gives total cost

$$H(n) + Q(n)S(n)^{O(1)}2^{O(K(n))}. \tag{15}$$

If Q, S, H are bounded by $\exp((\log(n+2))^{O(1)})$ and $K(n) = (\log(n+2))^{O(1)}$, the total is quasi-polynomial. In particular, $K(n) = O(\log n)$ gives a polynomial local parameter factor, while $K(n) = O((\log n)^2)$ gives $n^{O(\log n)}$ locally.

Proof. Apply Theorem 5.2 to each charged local call, including any whose answer is not subsequently used. Sum the resulting upper bound over $Q(n)$ calls and add $H(n)$. Binary path products, when authorized by (iv), have the bit bounds of Theorem 6.2. Products and sums of a fixed number of quasi-polynomial factors remain quasi-polynomial. The two stated specializations follow from exponentiation. □

The premises of Theorem 10.1 are *not established by this article*. In particular, no theorem here bounds $K(n)$ along every admissible unknot hierarchy, controls the total number of repairs, or proves that the retained rows always admit a cheap geometric completion. The current raw-cut construction and our local optimizer address different parts of that programme. The $L(g+1)^L$ iteration expression in Lackenby's Section 9 is a useful reminder that bounding one stage or one hierarchy length does not automatically control all repeated work [2].

11 Executed verification and component measurements

11.1 Environment and independence of the checks

The recorded environment is CPython 3.13.5 on Linux x86-64. The algorithms and tests use no external Python packages. All raw counts, samples, seeds, and result files accompany the packet. The unit-test log records 50 passing test methods in 0.353 seconds. The test methods include internal random loops, so the number 50 is not a count of distinct mathematical inputs.

The independent finite oracles deliberately use different formulations. One enumerates subsets of rows and repeatedly scans for private columns. Another enumerates arbitrary positive unit pivots in the saturated three-value calculus. The transport oracle executes literal uncapped positive fill. Path checks enumerate binary edge-selection strings. Forest checks enumerate every spanning tree of the chosen K_4 graph. None of these reference oracles uses the block/path dynamic programme to establish its expected answer.

No maintained native test suite, real source-geometric corpus, hard-knot corpus, or end-to-end recognition timing was executed. Repository access was targeted through GitHub text reads; no full runnable checkout was available in the computation environment. The article's native-source observations are source review, not freshly rerun experimental evidence.

11.2 Exhaustive matrix-case roles

The exhaustive suite runs the following complete finite families:

Rows	Columns	Entry alphabet	Executed matrices
2	3	$\{0,1,2\}$	729
3	3	$\{0,1\}$	512
3	3	$\{0,1,2\}$	19,683
2	4	$\{0,1,2\}$	6,561
3	4	$\{0,1\}$	4,096
Total executed matrix-case roles			31,581

Some families overlap: the binary 3×3 matrices also occur within the ternary family. Accordingly 31,581 is a count of executed case roles, not a claim of 31,581 pairwise distinct matrices. Every case runs both with and without private prepeeling, giving 63,162 optimized plans. All agree with the independent subset optimum. A further 1,241 of these cases also agree with the all-positive-pivots oracle. The recorded exhaustive run took 6.243 seconds and found zero discrepancies.

11.3 Weighted, typed, and forest audits

The seeded randomized suite executes 1,000 weighted matrix comparisons, 500 eligible-pair-restricted comparisons, 500 source-label permutations, and 1,000 literal numerical transport comparisons. These roles intentionally overlap in part. It also checks 300 closed-walk contexts on K_4, comparing all sixteen spanning trees in each context, for 4,800 forest comparisons. In every context the maximum forest score equals the full-matrix score, and the constructed deferred forest is compatible with the selected pivots.

There are 240 complete literal path-table comparisons, varying path length, endpoint type, alternating eligibility, and occupied/free potentials. Their full power proofs also replay. Unit tests reject altered table values, altered product references, wrong repetition lengths, malformed labels, parity violations, and capacity violations. The randomized suite found no discrepancies and took 1.028 seconds in the recorded run.

A large explicit cactus with 1,000 four-cycle blocks and 2,000 weighted rows gives matched weight 1,002,000 and deferred weight 500,500, exactly as Theorem 9.1 predicts. Its maximum block rank is one despite total cycle rank 1,000. The recorded optimizer uses 6,000 path states and tests 5,000 capacity-compatible kernel assignments.

The large symbolic cycle has half-length $N = 2^{1024}$. Its producer uses 2,049 matrix products and the proof contains 2,052 nodes. The saved JSON packet has 5,141,912 bytes including its final newline. Independent full-table replay succeeds, and the exact result is $\mathrm{OPT} = 2^{1024} - 1$, $\mathrm{def} = 1$. This test does not instantiate its exponentially many vertices.

11.4 Paired matrix-optimization measurements

Each benchmark has one untimed complete query per arm, followed by seven rounds with seeded shuffled arm order. An identical reference/reference arm supplies an A/A control. Garbage collection occurs before but outside the measured region. All samples finish and all compared answers agree; no speedup is extrapolated from a timed-out run.

The matrix benchmark compares the new optimizer, including normalization, witness extraction, its internal replay, and an additional independent feasibility replay, against the increasing-mask subset oracle. Both return the same scalar optimum. The new arm does extra witness work; the reference is nevertheless an intentionally exponential enumerator, not a competitive cactus-specific algorithm and not the maintained ordered-interface solver.

Rows	New (ms)	Reference (ms)	Paired ratio	A/A
2	0.249	0.044	0.182	1.036
4	0.338	0.078	0.226	0.943
8	0.479	1.121	2.341	0.879
12	0.776	26.690	35.631	1.017
16	0.766	625.572	808.588	1.014

Table 1: Unit-weight connected cactus matrices. “Paired ratio” is the median reference/new ratio; values above one favour the new arm. A/A is the median identical-reference ratio.

The two-row and four-row cases regress substantially because the generic block machinery costs more than tiny subset enumeration. On sixteen rows, the measured medians are 0.766 ms versus 625.572 ms, with median paired ratio 808.588. This illustrates the difference between a linear-block method and the specified exponential oracle; it does *not* establish superiority over known linear-time cactus algorithms [3]. The A/A ratio at eight rows is 0.879, showing that these short measurements have non-negligible timing variability. No claim of precise hardware-independent constants is intended.

11.5 Equal-query passive transport measurements

The transport benchmark uses proposition 8.3 with $p = s$. Input and the already selected valid plan are prepared outside timing. Both arms independently verify the supplied plan. The new arm constructs and validates the circuit, evaluates the sum of outside entries in every passive row, and serializes these answers. The reference arm performs literal positive fill, computes the same sums, and serializes the same answers. Neither arm serializes an expanded residual matrix as its final answer.

$p = s$	New (ms)	Reference (ms)	Paired ratio	A/A
2	0.134	0.095	0.693	1.038
8	0.165	0.113	0.684	1.009
32	0.284	0.224	0.787	0.995
128	0.724	1.359	1.928	1.002
512	2.836	19.738	6.973	0.989

Table 2: The same complete passive-row sum query, via a shared circuit or explicit positive fill. Reference/new ratios include construction, checking, evaluation, and answer serialization.

The circuit is slower for $p = s \leq 32$. At $p = s = 512$, the explicit residual has 262,144 nonzero entries while the source has 1,025; the measured query medians are 2.836 ms for the circuit and 19.738 ms for literal fill, with paired ratio 6.973 and A/A 0.989. This is a component-level synthetic improvement supported both by the output-size theorem and equal-query measurements. It is not a measured speedup of an actual knot-recognition call.

11.6 Complete binary-power queries

The symbolic benchmark times construction, independent full-table replay, and serialization together. It includes seven complete samples after one warmup. There is no explicit-expansion comparison at infeasible sizes.

$\log_2 N$	Products	Serialized bytes	Complete query (ms)
16	33	9,103	1.052
64	129	45,255	3.729
256	513	398,355	15.415
1024	2049	5,141,911	77.855

Table 3: Binary cycles with $2N$ edges. All arithmetic and proof data are exact. Serialized bytes exclude the final newline used in the saved audit packet.

For $N = 2^{1024}$ the complete symbolic query has median 77.855 ms. The growth in certificate bytes is important: even though the number of matrix nodes is linear in $\log N$, their integer entries grow in bit length. This is consistent with the quadratic-in-log N scalar-data bound, not a constant-size-certificate claim.

12 Further research questions and concrete next experiments

1. Native block-rank census. Measure the maximum individual block rank before and after forced peeling on authenticated source cores, especially the cases that stall in the maintained raw criterion. Record the whole block-rank distribution, not only its maximum. Does small κ explain any native successes that the current specialized regimes miss?

2. Geometric restrictions on cycle-bearing blocks. Which incidence blocks can arise from admissible marked handle cores after the actual parallelity reductions? Prove a bound in terms of source tetrahedra, boundary-pattern complexity, or a hierarchy potential; alternatively construct authentic families with large blocks. Arbitrary nonnegative matrices alone cannot settle this question.

3. A repeat-safe exporter from affine cut data. Construct a source checker that recognizes a repeated alternating incidence path directly in the binary-affine cut representation. It must establish fresh internal identities, endpoint parity, complete edge ownership, and descendant-context separation. A numerical period in the counts is insufficient.

4. Compressed geometric pivot traces. A binary path table certifies a score but does not emit an exponential flat plan. Develop a straight-line geometric trace language with source-bound semantics, efficient random access, and composition rules. Prove that each compressed instruction transports the actual attaching curves and boundary pattern.

5. Ordered passive transport. Lift Equation (12) from commutative counts to ordered words or ribbon attachments without losing embedding information. Identify operations for which shared donor circuits remain polynomially encoded, and establish a separate complexity bound for their geometric verifier.

6. Repairing the optimal stopping residual. The cactus barrier leaves one row per block. Which additional embedded slides, changes of charts, or verified disc operations resolve these rows in genuine handlebody cuts? A useful result would give both a repair construction and a decreasing potential, rather than merely a larger search space.

7. Hybrid width/rank selection. Combine the current ordered-incidence engines with the cycle-block solver block by block. Include the cost of decomposition discovery and certificate translation. Determine whether the best practical criterion depends on b, width, path count, eligible-edge density, or numerical weight distributions.

8. Better dependence on block rank. Can terminal-capacity constraints and acyclicity be solved more efficiently than enumerating all six-state path combinations? A direct dynamic programme on the branching skeleton might improve the constant base. Any lower-bound claim should distinguish cycle rank from treewidth and from the number of degree-three vertices.

9. Minimality of the interface. Determine which of the six path states are distinguishable by compatible bipartite contexts under weighted optimization. Are fewer states sufficient when endpoint types, eligibility, or descendant potentials satisfy additional native constraints? A minimality theorem would make the compression interface conceptually sharper.

10. Independently checkable general optimality. The explicit plan checker currently certifies feasibility, while optimality relies on the producer's complete dynamic programme. Export local block tables and all-candidate upper-bound evidence, as done for binary powers, so a separate verifier can certify the entire optimum. Quantify the certificate-size tradeoff.

11. Secondary forest objectives. After selecting pivots, choose a compatible forest minimizing an independently meaningful geometric cost. Simple additive edge costs admit a spanning-forest optimization in the complement; coupled costs involving later attaching-curve complexity need new analysis. Decide whether optimizing pivots and such a forest jointly preserves a useful parameterized bound.

12. Incremental block updates. A hierarchy may change only a small region of an incidence graph. Can source-bound block decompositions and path tables be updated incrementally, with persistent witnesses and exact invalidation? The bound must charge all affected ancestors and any new large block formed by the update.

13. Numerical observers and integral invariants. Use transport circuits for selected exact observables while retaining a separate faithful geometric representation. Identify observables that safely prune candidate geometric moves. Do not infer ball status from vanishing homology or from the absence of a matrix obstruction.

14. Native performance validation. Run the adapter only behind authenticated source reconstruction and geometric replay. Compare complete queries against current maintained paths, including small regressions, unsuccessful attempts, every resource refusal, serialization, and downstream residual processing. The synthetic gains here are a reason to conduct this experiment, not a substitute for it.

15. Formal verification. The six-state lemma, block-capacity composition, stopping-set persistence, and mod-two coforest argument are compact candidates for Lean formalization. Formalize the source-private contract first so that the theorem's relation to genuine handle geometry remains explicit.

16. A global amortized hierarchy potential. The decisive next theoretical step is a potential controlling all successor constructions, resets, residual repairs, and certificate growth. Relate that potential to a bound on block rank and the costs of generating repeat templates. Only a theorem of this kind, combined with complete geometric operations, can discharge the premises of Theorem 10.1.

13 Conclusion

The local obstruction is not the total size of a long incidence path: its relevant matching and alternating-cycle behavior fits into six states. Nor is it the total number of easy cycle blocks: articulation joins need only one capacity bit. The resulting exact optimizer has single-exponential dependence on the largest individual block cycle rank, and supplied repeated paths admit polynomial encoded-time score certificates.

Weighted partial deferred forests eliminate a separate primal-tree search under the uncapped source-cycle contract. Positive transport circuits avoid a different bottleneck, namely the needless materialization of passive fill when only linear queries are required. These three layers compose mathematically while retaining distinct output and trust contracts.

The residual barrier is equally important: even rank-one blocks can leave linearly many genuine two-handles. Thus the article supplies a sharper, tested local engine and explicit geometric research obligations. It neither erases those obligations nor confuses local optimality with a completed quasi-polynomial recognition proof.

A Reproduction and API contracts

Run from the unpacked packet root:

```
python -m unittest discover -s tests -v
python scripts/audit.py --suite exhaustive
python scripts/audit.py --suite randomized
python scripts/audit.py --suite large
python scripts/benchmark.py --suite all
python scripts/make_tables.py
python -m cycleblocks examples/weighted_cactus.json --transport
sh build.sh
```

The audit and benchmark commands overwrite their default result files. Use their `-output` option to preserve the distributed evidence. The large suite also writes its example certificate; run a reproduction on a copy of the packet when exact delivered evidence is to be retained.

The main Python interfaces are:

Interface	Contract
`optimize_counts`	Exact explicit matrix optimum on completion; resource refusal is inconclusive.
`verify_plan`	Independent source-private feasibility and score replay, not global optimality.
`periodic_path_table`	Complete symbolic table for the declared fresh alternating template.
`verify_power_certificate`	Independent all-candidate product and type/length replay.
`transport_circuit`	Matrix-bound positive numerical transport; no embedded-curve data.
`evaluate_circuit`	Exact integer linear observable on every retained passive row.
`deferred_forest`	Uncapped parity and graph-complement construction; no source authentication.

A minimal weighted example has rows supported on (c_0, c_1) and (c_0, c_2) in repeated pairs. The supplied example has weights $(2, 7, 11, 3)$, so it cancels weight eighteen and retains weight five. Its explicit plan and positive circuit are in `examples/weighted_cactus_plan.json`.

B Claim ledger and source audit

The following table summarizes the evidentiary status rather than introducing further results.

Claim	Status and boundary
Positive/private equivalence	Inherited project mathematics, reproved under unrestricted positive-unit rules. Restricted eligibility is supported only for source-private orders.
UR matching connection	Classical concept and alternating-cycle characterization; row weights and unit eligibility stated explicitly.
Six-state path kernel	Proved here with exact context potentials, terminal capacities, and directed-cycle preservation.
Maximum-block-rank bound	Proved here: $S^{O(1)}2^{O(\kappa)}$, with $3b-2$ paths in a rank-$b \geq 2$ block. No global native κ bound.
Cactus tractability	Classical for maximum-cardinality UR matching; weighted/eligible implementation is a specialization, not a first-in-literature claim.
Binary powers	Exact for declared repeat-safe templates. Full-table optimality replay implemented. No general native exporter or expanded trace.
Partial weighted forests	Consequence extending the inspected full deferred-forest argument. Requires original uncapped row-cycle parity.
Positive transport DAG	Exact commutative count transport and linear observables. Ordered geometric attachments not encoded.

Claim	Status and boundary
Experimental results	Executed standalone tests, finite audits, abstract graph contexts, and synthetic component benchmarks. Not native knot evidence.
Quasi-polynomial recognition	Conditional accounting only. Global structural, repair, marked-transport, and completeness premises remain unproved here.

The source review used GitHub reads of the repository README, the maintained raw-cut section, the first 240 lines of `core_handlebody.py`, report 105's README, report 108's claims, report 109's README, and report 110's claim/integration text exposed by its placement commit. The exact maintained source pin is

`d9456c5bdb94820bfa9e480e9ed73c10a769e59a`.

That commit places report 110; placement is not a soundness review. Earlier unpinned README and cut-section reads were contextual. The implementation contract is tied to the pinned core source. No claim is made to have audited every incoming archive or every report in the large repository. A new intake should check subsequent changes before adapting the code.

References

[1] M. Lackenby, *Unknot recognition in quasi-polynomial time*, March 2021 lecture slides linked from the author's Oxford webpage. https://people.maths.ox.ac.uk/lackenby/quasipolynomial-talk-oxford.pdf. Accessed 12 October 2026. This is the announced complexity target, not a cost analysis supplied by this article.

[2] M. Lackenby, *Incompressible surfaces, hierarchies and unknot recognition*, arXiv:2607.23350v1, 25 July 2026, 54 pages; especially Sections 9–10. https://arxiv.org/abs/2607.23350.

[3] M. C. Golumbic, T. Hirst, and M. Lewenstein, *Uniquely restricted matchings*, Algorithmica **31** (2001), 139–154. https://doi.org/10.1007/s00453-001-0004-z. Bibliographic and abstract information checked against the authors' institutional publication records.

[4] J. Chaudhary, I. Sau, and M. Zehavi, *A parameterized perspective on uniquely restricted matchings*, arXiv:2508.12004v1, 16 August 2025. https://arxiv.org/abs/2508.12004.

[5] V. Reshetnikov, *ProveIt*, pinned revision stated in Appendix B, file `Topology/UnknotRecognition/fast/fastunknot/core_handlebody.py`. Inspected source extraction, full private queue, and deferred-forest routines. https://github.com/VladimirReshetnikov/ProveIt.

[6] ProveIt maintained synthesis, *Full three-dimensional normal cuts in affine cell families*, file `Topology/UnknotRecognition/synthesis/cut_cells.tex`; repository README checkpoint, inspected 12 October 2026.

[7] ProveIt report 105, *Deferred forests and exceptional incidence*, README and stated full primal-forest theorem, at the pinned revision. `Topology/UnknotRecognition/reports/105/README.md`.

[8] ProveIt report 108, activation-forest claim inventory, at the pinned revision. `Topology/UnknotRecognition/reports/108/CLAIMS.md`. This report was inspected to distinguish the exceptional-support programme from the present block/path approach.

[9] ProveIt report 109, *Optimal deferred-handle kernels via ordered incidence interfaces*, README at the pinned revision. `Topology/UnknotRecognition/reports/109/README.md`.

[10] ProveIt report 110, *Ordered pivot interfaces*, delivered claim and integration ledger exposed in placement commit `d9456c5bdb94820bfa9e480e9ed73c10a769e59a`. `Topology/UnknotRecognition/reports/110/CLAIMS.md` and `Topology/UnknotRecognition/reports/110/INTEGRATION.md`.

[11] ProveIt incoming-report procedure, `docs/incoming/README.md`, inspected 12 October 2026. Intake and native promotion are separate operations.